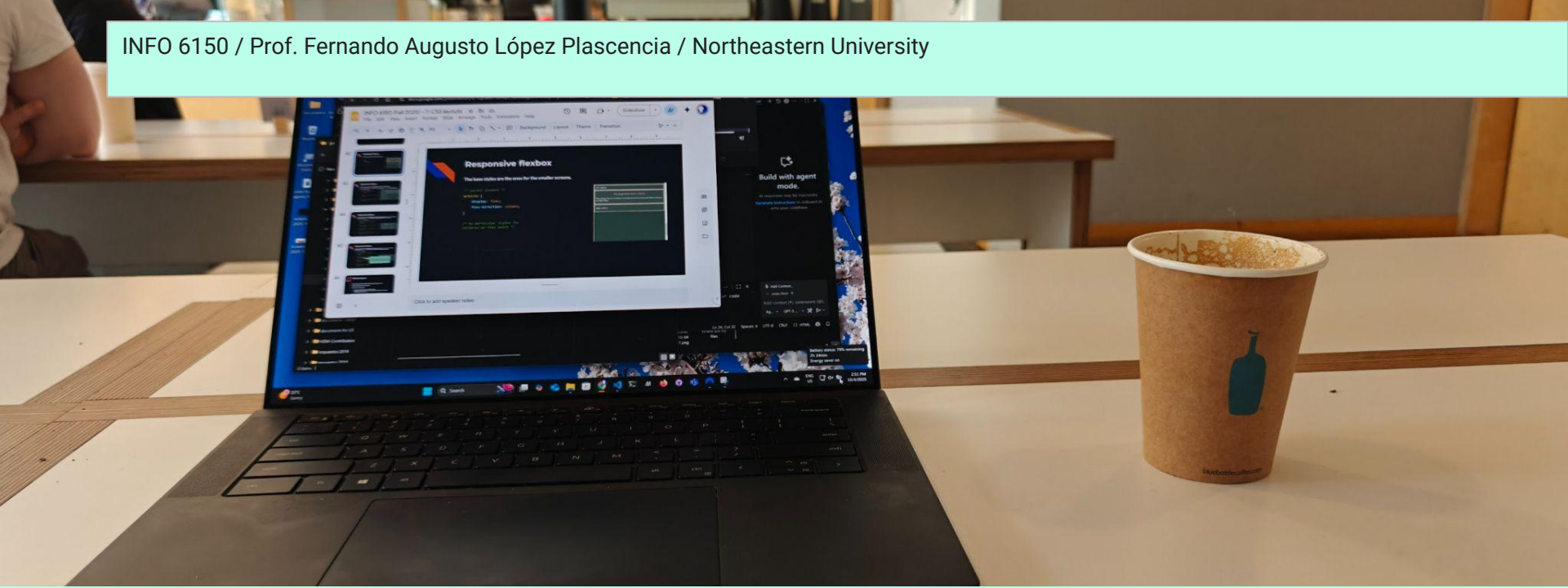
INFO 6150 / Prof. Fernando Augusto López Plascencia / Northeastern University




Week 6

# JavaScript Basics

# The website development process

- Discovery and planning
  - What is the problem?
  - What are we doing to address it?
- Development
  - Actually doing the things
- Deployment
  - Shipping
- Maintenance
  - Keeping up and measuring success

We're still here… but taking a pause

# In this lesson:

- The secret to learn JS
- Data Types, Constants, Variables
- Arrays and their traversal
- Objects
- JSON
- Functions
- Scope
- Appendix: Making good functions
- Appendix: Keyed collections

# What is the secret to learning JS?

Some courses or professors will tell you there is no trick to learn JS. This is false. Read on...

# What is the secret to learning JS?

Some courses or professors will tell you there is no trick to learn JS. This is false. Read on...

The secret is:

# What is the secret to learning JS?

Some courses or professors will tell you there is no trick to learn JS. This is false. Read on...

The secret is:

## Repetition!

Effectively learning JS, or programming in general, is like learning a new language.

- Understand the concepts
- Practice, practice, practice until you build muscle memory

There is no shortcut to this. The more you think by yourself, the more you will understand.

# JavaScript does not need to be compiled

Most languages are **compiled**: they need to be converted to machine code by a compiler program before they can be run.

In contrast, JavaScript is executed on the spot. The JS executor reads and executes line by line.

# Embedding JS in a page

Two ways:

- Insert directly inside a <script> tag:

```
        <script>
            console.log('This script is directly embedded into the page.')
        </script>
```

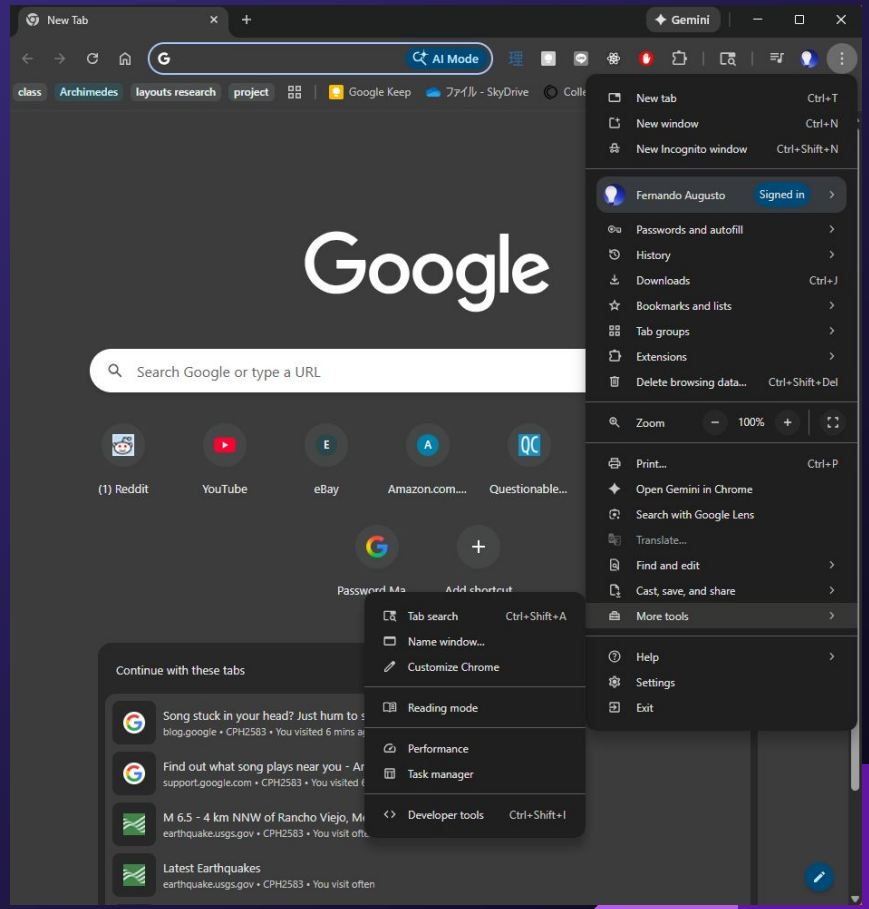
- Link an external script through `<script src="path/to/file.js">`:

```
<html>
    <head>
        <script src="script.js"></script>
    </head>
```

# Quickly running JS

Use the console in your browser's Developer Tools to quickly run JavaScript instructions.

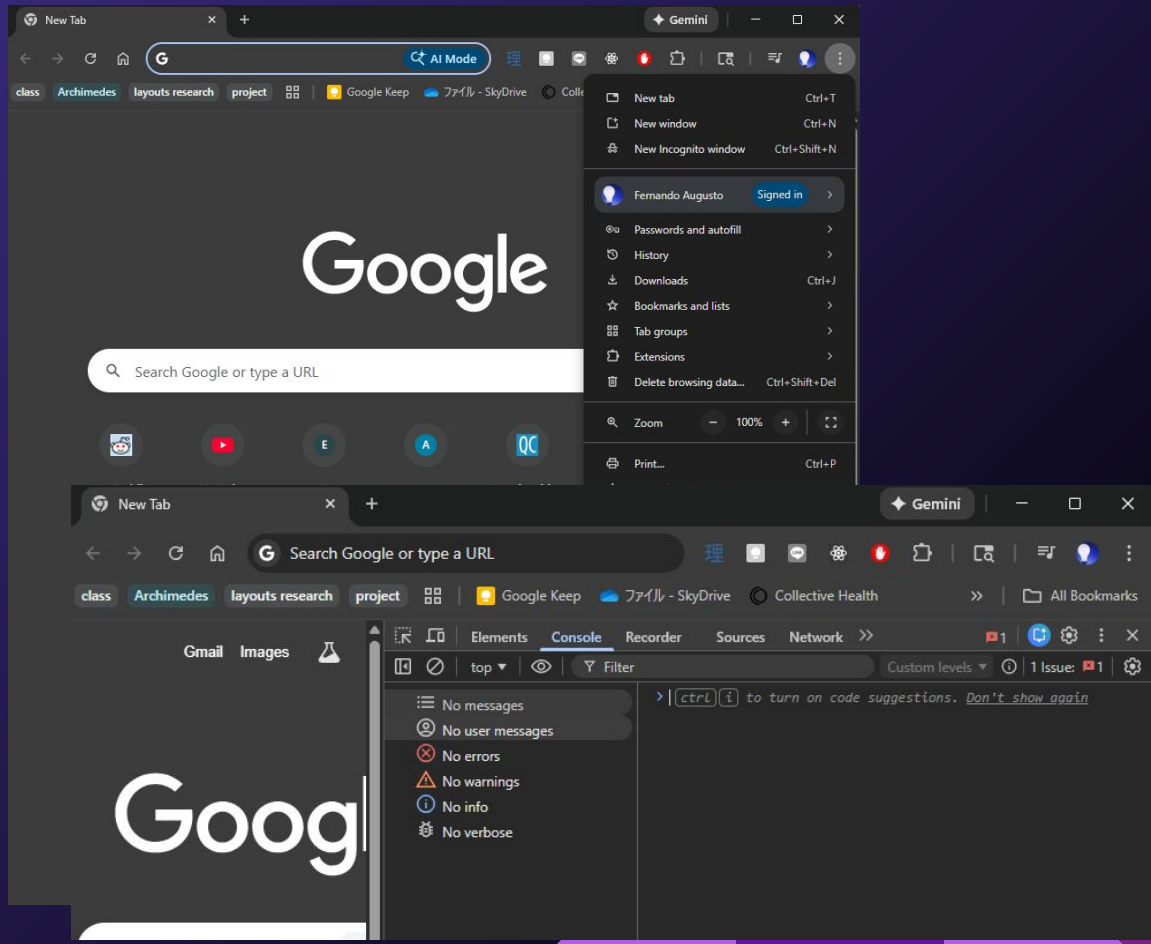
Use this to test the code in these slides.



# Quickly running JS

Use the console in your browser's Developer Tools to quickly run JavaScript instructions.

Use this to test the code in these slides.



# Data Types, Variables, Constants

# Data Types

Main **primitive** data types with examples:

| | | |
|---|---|---|
| Number | `2, -4.53, 2e10, NaN, Infinity, -Infinity` | Represents both integer and real numbers. |
| String | `"Hi", "123", "a",""` | Character strings of any size. |
| Boolean | `true, false` | Used for boolean operations and comparisons. |
| Null | `null` | Means: intentionally without a value |
| Undefined | `undefined` | Means: the variable has not been initialized. Absence of a value. |

# Data Types

JavaScript has the following **primitive** data types (from Mozilla):

| Type | `typeof` return value | Object wrapper |
|---|---|---|
| Null | `"object"` | N/A |
| Undefined | `"undefined"` | N/A |
| Boolean | `"boolean"` | `Boolean` |
| Number | `"number"` | `Number` |
| BigInt | `"bigint"` | `BigInt` |
| String | `"string"` | `String` |
| Symbol | `"symbol"` | `Symbol` |

# Data Types

…plus the following complex types:

- Array
- Object
- Keyed collections: Map, Set, WeakMap, WeakSet

that we'll explore in a bit.

# Variables and Constants

Data gets stored in memory positions represented by **variables** and **constants**.

Declare a variable with the keyword `let`.

To “declare” the variable means to create it. You may or may not assign a value to it when you declare it; if you don’t give it a value, it will contain `undefined`.

```
> let myVar = 1;
<· undefined
> myVar
<· 1
> myVar = 2;
<· 2
> myVar = myVar + 1;
<· 3
>
```

```
> let anotherVar;
<· undefined
```

# Variables and Constants

Data gets stored in memory positions represented by **variables** and **constants**.

Declare a variable with the keyword `let`.

To “declare” the variab
create it. You may or n
a value to it when you declare it, if
you don’t give it a value, it will contain `undefined`.

```
> let myVar = 1;
<· undefined
> myVar
<· 1
> myVar = 2;
<· 2
> myVar = myVar + 1;
<· 3
>
```

With numbers you can use [arithmetic operators](#).

```
> let anotherVar;
<· undefined
```

# Variables and Constants

A constant is similar to a variable, but once it’s created, its value cannot change.

Declare a constant with the keyword `const`.

```
> const myConst = "Fernando"
<· undefined
> myConst = "Lopez"
✕ ▶Uncaught TypeError: Assignment VM484:1
  to constant variable.
      at <anonymous>:1:9
```

# Variables and Constants: when to use each?

Use a **variable** if the value will change later in the program.

Otherwise, use a **constant**.

```
const userName = window.prompt('Please enter your name.');
window.alert(`Salutations, ${userName}!`)
```

It is good practice to use constants whenever you can.

# Variables and Constants: when to use each?

Use a **variable** if the value will change later in the program.

Otherwise, use a **constant**.

```
const userName = window.prompt('Please enter your name.');
window.alert(`Salutations, ${userName}!`)
```

It is good practice to use constants whe

This is a way of putting the value of a variable or constant inside of a string. More on this later.

# Dynamic Typing

In most programming languages, you define the type of your variables / constants and they stay with that type. Trying to assign a value with the wrong type would be an error.

Not with JavaScript. Its types are **dynamic**: the variables can be assigned values of all types.

```
> let someVar = "hi";
<· undefined
> someVar = 3.4;
<· 3.4
> someVar = true;
<· true
>
```

# Weak Typing

In most programming languages, you cannot do operations with values of different types; you need to convert them or you get an error.

JavaScript's types are weak: instead of an error, JS will try to convert the values implicitly.

```
> anotherVar = 355;
<· 355
> anotherVar = "2" + anotherVar;
<· '2355'
```

# Weak Typing

This conversion can lead to errors when JS does a conversion that you did not expect it to do.

Can you guess what happened here?

```
> someVar = true;
<· true
> someVar = 32 + someVar;
<· 33
```

# Weak Typing

This conversion can lead to errors when JS does a conversion that you did not expect it to do.

Can you guess what happened here?

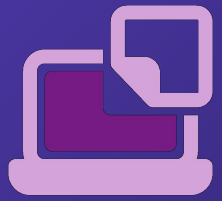
The answer: the boolean value `true` got converted to the number value `1` before doing the sum.

```
> someVar = true;
<· true
> someVar = 32 + someVar;
<· 33
```

# Knowing a value's data type

The `typeof` operator returns the type of a value, as a string.

```
> typeof 1
<· 'number'
> typeof "abc"
<· 'string'
> const test = "23" + 1;
<· undefined
> test
<· '231'
> typeof test
<· 'string'
> typeof {}
<· 'object'
> typeof true
<· 'boolean'
> typeof xyz
<· 'undefined'
```



Created by HideMaru
from Noun Project

# A challenge!

Create an HTML page, then link a script to it.

The script must:

- Ask the user for their name (use `window.prompt`)
- Greet the user using that name

Store the name in a constant.

Icon by HideMaru for Noun Project

# Numbers

# Some notes on numbers

There is no separate type for floats and integers: it's all `Number`.

`Number` is a 64-bit value: it can represent values from $(2^{53} - 1)$ to $-(2^{53} - 1)$. For bigger or smaller values you can use `BigInt` instead.

JS

Copy

```js
255; // two-hundred and fifty-five
255.0; // same number
255 === 255.0; // true
255 === 0xff; // true (hexadecimal notation)
255 === 0b11111111; // true (binary notation)
255 === 0.255e3; // true (decimal exponential notation)
```

From Mozilla

# NaN

The value `NaN` (or `Number.NaN`) is a “number error”. `NaN` means “not a number”.

You can get `NaN` from invalid math operations.

```
> NaN
<· NaN
> typeof NaN
<· 'number'
```

```
> const result = "hello" / 2;
  console.log(result); // NaN
  NaN                                  VM4043:2
```

# NaN

Because NaN has the `Number` type, the more reliable way to check if a numeric value has `NaN` is to use the `Number.isNaN()` function.

```
> const xyz = NaN
<· undefined
> xyz
<· NaN
> isNan(xyz)
✕ ▶Uncaught ReferenceError:                VM3936:1
  isNan is not defined
      at <anonymous>:1:1
> Number.isNaN(xyz)
<· true
>
```

# Other special values

...include:

```
> 100 / 0
<· Infinity
> -100 / 0
<· -Infinity
> Number.MAX_VALUE
<· 1.7976931348623157e+308
> Number.MIN_VALUE
<· 5e-324
```

# Strings and String Operations

# Strings are special

In JavaScript, the base type is the string. This is because it was made to be a programming language for the web, and a web page is made of strings.

This means any value can be converted to a string. For most of them, you just need to call .toString(). For example, for a number:

```
> (20).toString()
<· '20'
```

# Strings in JavaScript

We don't have a "character" type in JS; we only have String.

Create a string using double quotes (" "), single quotes (' ') or backticks (` `).

- Backtick strings have special behaviors:
  - They can span multiple lines
  - They can include templates (more on that in a bit)

```
> "asdf
    asdf"
Uncaught SyntaxError: Invalid or unexpected token
> `asdf
  asdf`
<· 'asdf\nasdf'
```

# Operations with strings

Getting the string length: use .length

```
> "abc".length
<· 3
```

Concatenating (sticking strings together): use the + operator…

```
> "abc" + "def"
<· 'abcdef'
```

…or templates: in a backticked string, use ${} to insert the value of another variable

```
> const str = "abc"
<· undefined
> `${str}def`
<· 'abcdef'
```

# Control Structures

# Conditions with if

Use the if keyword to create conditions. It has this structure:

```
if (/* condition */) {
    /* code to execute if condition is true */
} else {
    /* code to execute if condition is false */
}
```

# Conditions with if

Use the if keyword to create conditions. It has this structure:

```
if (/* condition */) {
    /* code to execute if condition is true */
} else {
    /* code to execute if condition is false */
}
```

The **condition** can be anything that can be true or false.

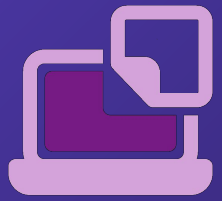
# Conditions with if

Use the if keyword to create conditions. It has this structure:

```
if (/* condition */) {
    /* code to execute if condition is true */
} else {
    /* code to execute if condition is false */
}
```

Everything inside a pair of brackets {} is called a **block.**

The condition can be anything that can be true or false.



Created by HideMaru
from Noun Project

# Conditions with if

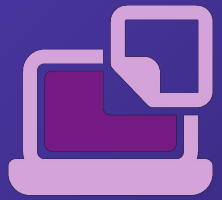
Write the following on your script and run it:

```
const userAge = window.prompt("Please enter your age:");
if (userAge >= 18) {
    console.log("You are an adult.");
} else {
    console.log("You are a minor.");
}
```

Try it with different ages.

# Conditions with if

We can also link up the `if` statements with `else if` clauses for more complex checks.



Created by HideMaru
from Noun Project

# Conditions with if

Try this code. Test it by writing your name, not writing your name, and by cancelling the prompt.

```
const userName = window.prompt("Please enter your name:");
/* If the user cancels the prompt, userName will be null */
if (userName === null) {
    console.log("You cancelled the prompt.");
} else if (userName === "") {
    console.log("You did not enter a name.");
} else {
    console.log(`Welcome, ${userName}!`);
}
```

# Conditions with switch

If you want to compare the same value many times…

```
let color = '';
if (fruit === 'apple') {
    color = 'red';
} else if (fruit === 'banana') {
    color = 'yellow';
} else if (fruit === 'grape') {
    color = 'purple';
} else {
    color = 'unknown';
}
```

# Conditions with switch

...you can use the `switch` operator instead.

In switch you choose which case to follow depending on the value of a single statement (such as a variable or constant). The value does not need to be a boolean.

```
switch(fruit) {
    case 'apple':
        color = 'red';
        break;
    case 'banana':
        color = 'yellow';
        break;
    case 'grape':
        color = 'purple';
        break;
    default:
        color = 'unknown';
}
```

# Conditions with switch

...you can use the `switch` operator instead.

In switch you choose which case to follow depending on the value of a single statement (such as a variable or constant). The value does not need to be a boolean.

```
switch(fruit) {
    case 'apple':
        color = 'red';
        break;
    case 'banana':
        color = 'yellow';
        break;
    case 'grape':
        color = 'purple';
        break;
    default:
        color = 'unknown';
}
```

Each case ends with “break”.

If you forget the “break” the execution will continue in the next case!

# Conditions with switch

...you can use the `switch` operator instead.

In switch you choose which case to follow depending on the value of a single statement (such as a variable or constant). The value does not need to be a boolean.

```
switch(fruit) {
    case 'apple':
        color = 'red';
        break;
    case 'banana':
        color = 'yellow';
        break;
    case 'grape':
        color = 'purple';
        break;
    default:
        color = 'unknown';
}
```

default is the same as “else”: if no other case was applicable, the default happens.

# Conditions with switch

...you can use the `switch` operator instead.

In switch you choose which case to follow depending on the value of a single statement (such as a variable or constant). The value does not need to be a boolean.

```
switch(fruit) {
    case 'apple': {
        color = 'red';
        break;
    }
    case 'banana': {
        color = 'yellow';
        break;
    }
    case 'grape': {
        color = 'purple';
        break;
    }
    default: {
        color = 'unknown';
    }
}
```

It's not required, but a good idea to use blocks.

# Logical Expressions

The comparisons inside the if() are **logical expressions**. They are conditions that return `true` or `false`.

Use relational operators such as <, >, <=, >= and equality operators such as ==, ===, != and !== to compare between values.

!= and !== means “is not equal to”.

```
> const a = 1
<· undefined
> a > 1
<· false
> a >= 1
<· true
> const b = 10
<· undefined
> a < b
<· true
> let c;
  if (a < b) {
      c = a;
  } else {
      c = b/2;
  }
  console.log("value of c: ", c);
  value of c:  1                    VM1680:7
```

# Logical Expressions

Do not use == or !=. They automatically convert value types and this can lead to errors.

Always use === and !== instead.

```
> 1 === "1"
<· false
> 1 == "1"
<· true
```

# Logical Expressions

You can join conditions using the [binary logical operators](#) && (and) and || (or). They as you would expect:

```
true && false === false
true || false === true
```

```
const userName = window.prompt("Please enter your name:");
if (userName === null || userName === "") {
    console.log("I did not get your name.");
} else {
    console.log(`Welcome, ${userName}!`);
}
```

# Logical Expressions

Because of JS's weak typing, you can still use expressions that do not return true or false and they will convert:

Convert to false: `0`, `null`, `undefined`, `""` (the empty string), `Number.NaN`. These are called **"falsy" values**.

Convert to true: everything else

```
> const a = 3;
<· undefined
> if (a) {
      console.log("the constant a has a value");
  }
  the constant a has a value        VM2719:2
```

# Some assignation shortcuts

When we put a value in a variable or constant it’s said we **assign** it.

Some assignations are so common that there are shortcuts to do them. We’ll see two of them.

# The tertiary operator (?:)

If you have to assign a value depending on a condition...

```
const num = window.prompt("Enter a number:");
let numberSign;
if (num > 0) {
  numberSign = '+';
} else {
  numberSign = '-';
}
console.log(`The number is
${numberSign}${Math.abs(num)}`);
```

# The tertiary operator (?:)

...you can use a **tertiary operator** instead.

The format of the tertiary operator is

```
condition ? expressionIfTrue : expressionIfFalse
```

Meaning: if the condition is true, the operator returns the first expression's value; if the condition is false, it returns the second one.

```
const num = window.prompt("Enter a number:");
const numberSign = num > 0 ? '+' : '-';
console.log(`The number is
${numberSign}${Math.abs(num)}`);
```

# The tertiary operator (?:)

This is also useful if you're checking if a value is **falsy** (for example: "", 0, etc.)

```
let message = "";
if (userName) {
    message = `Hello, ${userName}!`;
} else {
    message = "I do not know your name.";
}
```

becomes

```
const message = userName ? `Hello, ${userName}!` : "I
do not know your name.";
```

# The nullish coalescing operator (??)

Sometimes, you only want to check if there *is* a value - that is, if the value is not `null` or `undefined`.

```
// could be 0, which is falsy, but a valid value
const howMany = window.prompt("How many pounds have you lost?");

let lostPounds;
if (howMany === null) {
    lostPounds = 0;
} else {
    lostPounds = howMany;
}
console.log(`You have lost ${lostPounds} pounds.`);
```

# The nullish coalescing operator (??)

In these cases you can shorten things up with the `??` operator.

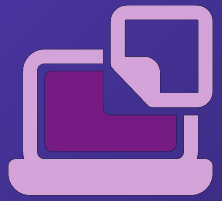
Its format is

```
value ?? defaultValue
```

where, if `value` is null or undefined, what's returned is the `defaultValue` instead.

```
// could be 0, which is falsy, but a valid value
const howMany = window.prompt("How many pounds have you lost?");

let lostPounds = howMany ?? 0;
console.log(`You have lost ${lostPounds} pounds.`);
```



Created by HideMaru
from Noun Project

# Conditions and logical expressions: Exercises

1. Write code that takes a numeric score from 0 to 100 and prints the corresponding letter score (A, B, C, D, F).
2. Write code that reads a month number (1: January, 12: December) and returns the season (Spring, Summer, Autumn, Winter).
3. Check if a user will get a discount. The user gets a discount if they’re a senior (ages 65 and up) OR are underage (ages 21 or less) OR if they’ve spent more than $1000.

Icon by HideMaru for Noun Project

# Objects

# Objects

An **object** is a **data structure** that symbolizes something complex via **keys** and **values** such as:

`username: “Fernando”;`

key

value

An object’s keys must be strings or numbers*.

*Or also `symbol`, but don’t worry about that one

```
const obj = {
  property1: value1, // property name may be an identifier
  2: value2, // or a number
  "property n": value3, // or a string
};
```

From Mozilla

# Objects

An object contains:

- **Attributes** - basically, other values.
  - They describe the object
- **Methods** - basically, functions that use the object data.
  - They describe what the object can do

```
const obj = {
  property1: value1, // property name may be an identifier
  2: value2, // or a number
  "property n": value3, // or a string
};
```

From [Mozilla]

# Objects

```
// An object.
// Separate object components using commas
const person = {
    name: "Fernando",
    age: 44, // age is just a number
    sayName: function() {
        console.log(this.name); // will print "Fernando"
        // "this" refers to the current object
    },
}
```

# Destructuring

Use this to more easily get data from within an object.

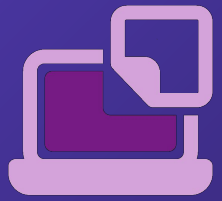
```
const { rates } = data; // this is the same as "const rates = data.rates"
```

You can destructure many elements at the same time; for example:

```
const { timestamp, base, rates } = data;
```

would create one constant for each one of those elements in the object.

If the element does not exist, you will get a value of `undefined`.



Created by HideMaru
from Noun Project

# Make some objects

Create objects that describe:

- A car
- A kitchen recipe
- A clothing article

Icon by HideMaru for Noun Project

# Objects and prototypes

JS objects are based on **prototypes.**

A prototype is a “hidden object” inside each object. It’s a way to do inheritance: for example, the `String` prototype contains all methods and attributes common to all strings, such as “abc”`.length` or “abc”`.toLocaleString()`.

```
> "abc".__proto__
<· ▾ String {'', anchor: ƒ, at: ƒ, big: ƒ, blink: ƒ, …} ℹ
    ▸ anchor: ƒ anchor()
    ▸ at: ƒ at()
    ▸ big: ƒ big()
    ▸ blink: ƒ blink()
    ▸ bold: ƒ bold()
    ▸ charAt: ƒ charAt()
    ▸ charCodeAt: ƒ charCodeAt()
    ▸ codePointAt: ƒ codePointAt()
    ▸ concat: ƒ concat()
    ▸ constructor: ƒ String()
    ▸ endsWith: ƒ endsWith()
    ▸ fixed: ƒ fixed()
    ▸ fontcolor: ƒ fontcolor()
    ▸ fontsize: ƒ fontsize()
```

# Classes

In Java and other OOP languages, you need to create a class in order to make an object out of it. In JS, while you don't **need** to use classes, you can.

In fact, some default objects in JS expect you to use class behavior.

For example, the class `Date` represents timestamps. Look at this code:

```
const bigDay = new Date(2019, 6, 19);
console.log(bigDay.toLocaleDateString());
if (bigDay.getTime() < Date.now()) {
  console.log("Once upon a time...");
}
```

Source: Mozilla

Here, `Date` is the name of a class; the keyword `new` creates a new object of that class.

# Classes

If you need to know what's the class of an object, don't use `typeof`: that will always return `'object'`. Instead, use `instanceof`

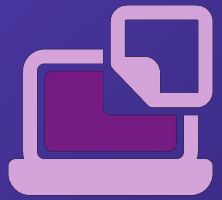
```
> const now = new Date()
<· undefined
> now
<· Sun Jan 04 2026 18:36:40 GMT-0800 (
> typeof now
<· 'object'
> now instanceof Date
<· true
> now instanceof Object
<· true
```

# Classes

You can think of a class as a “mold” to make objects. You make this mold once, then you can make as many objects from this mold as you want, and they will contain the same shape.

- Here, “the same shape” means: the same methods and the same (uninitialized) properties.
- In JS, this works by copying a reference of the parent class’ prototype to the objects.

Photo by April Miyako: https://www.pexels.com/photo/close-up-of-colorful-jellies-8191230/



Created by HideMaru
from Noun Project

# Making your own class

Copy this and paste in a file, then run. What happens?

Then, make a `Clothing` class based on the previous exercise, and an object of the `Clothing` class.

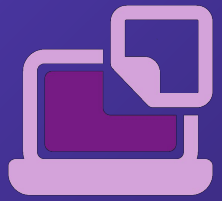
```
class Vehicle {
    constructor(make, model, year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }
}

let myCar = new Vehicle('Toyota', 'Corolla', 2020);
console.log(myCar);

// Extended class: it inherits make, model, and year from Vehicle
class Motorcycle extends Vehicle {
    constructor(make, model, year, hasSidecar) {
        super(make, model, year); // calls Vehicle constructor
        this.hasSidecar = hasSidecar;
    }
}

let myMotorcycle = new Motorcycle('Harley-Davidson', 'Street 750',
2019, false);
console.log(myMotorcycle);
```

Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

# More about classes

JS classes have other features, such as **static members**, **private fields**, and **getters (accessors)** that we won't cover right now. Read more about those here if you're interested: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_classes

Icon by HideMaru for Noun Project

# Objects are everywhere!

We said that objects inherit their behavior via prototypes.

But, also, In Javascript, **most values are objects** inheriting their behavior from a class even if they don't look like it.

For example:

- Strings inherit from the `String` class
- Numbers inherit from the `Number` class
- Objects manually created inherit from the `Object` class.

# JSON and object serialization

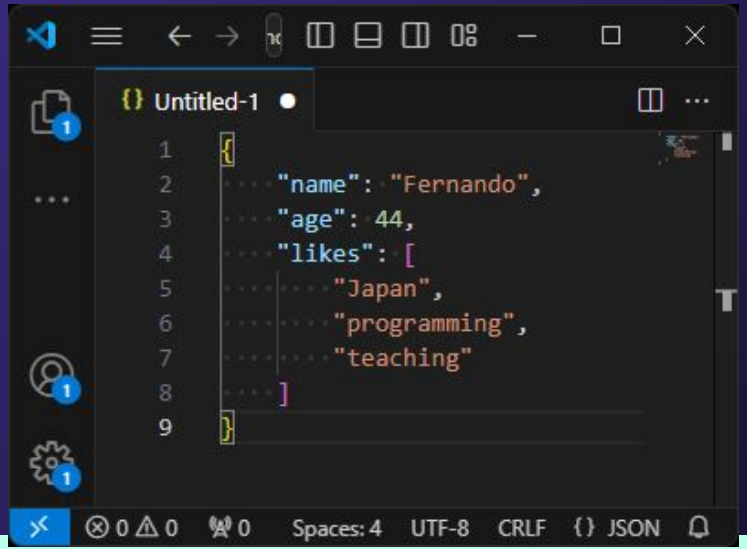
To “serialize” means to convert anything into a stream of bytes, which can then be transmitted or stored. The inverse process is to deserialize.

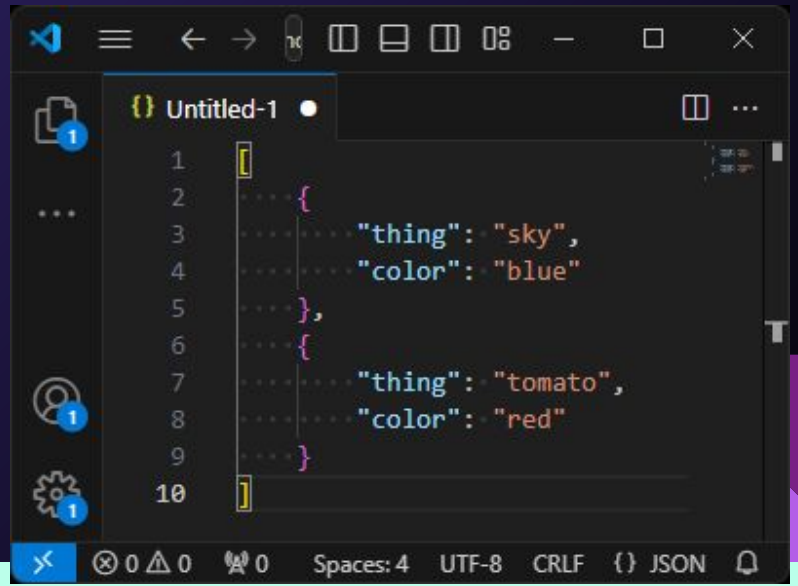
In a running program, objects are structures that live in memory. JavaScript solves serialization by converting them to **strings** with a standard format: JSON.

# What is JSON?

JSON is a standard for transmitting data in web applications, so it's important to get familiar with it.

The root of a JSON can be an object or an array.



# JSON serialization and deserialization

To serialize an object into a JSON string, you call `JSON.stringify()`.

To deserialize the JSON string into an object, you call `JSON.parse()`.

```
> const obj = {1: 'a', 2: 'd'}
<· undefined
> const str = JSON.stringify(obj);
<· undefined
> str
<· '{"1":"a","2":"d"}'
> const againObj = JSON.parse(str);
<· undefined
> againObj
<· ▶ {1: 'a', 2: 'd'}
>
```

# Warnings when using JSON

JSON is *delicate*. If you make a mistake when writing it, it won't work.

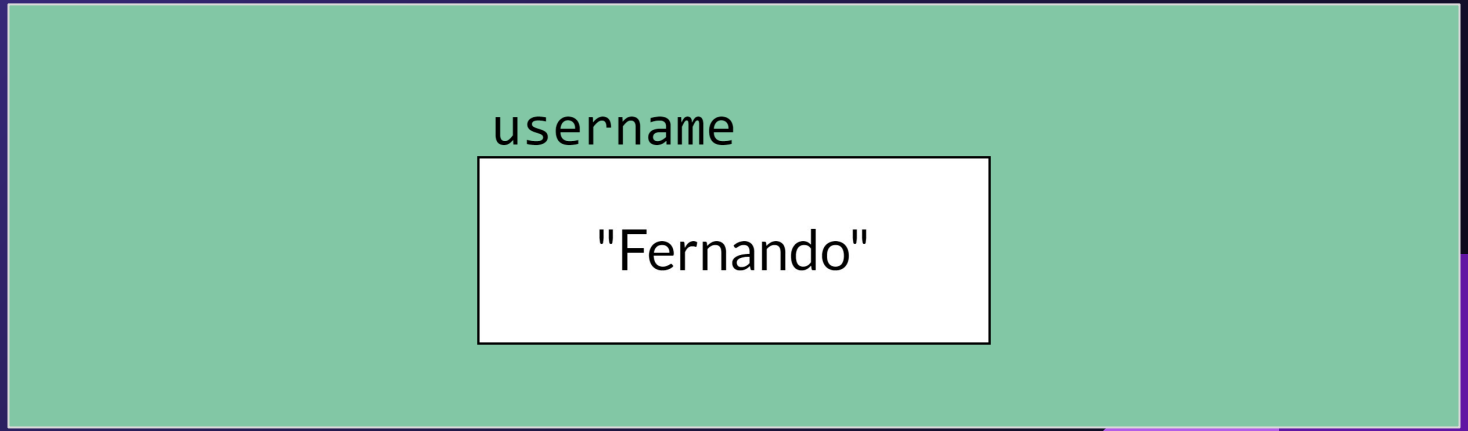
Use a JSON validator to make sure the JSON is fine! There are many, such as https://jsonlint.com/ , or https://jsonformatter.curiousconcept.com/# which will fix your JSON for you.

# Arrays

# Arrays

A variable stores one value. A constant is the same, but the value does not change once set.

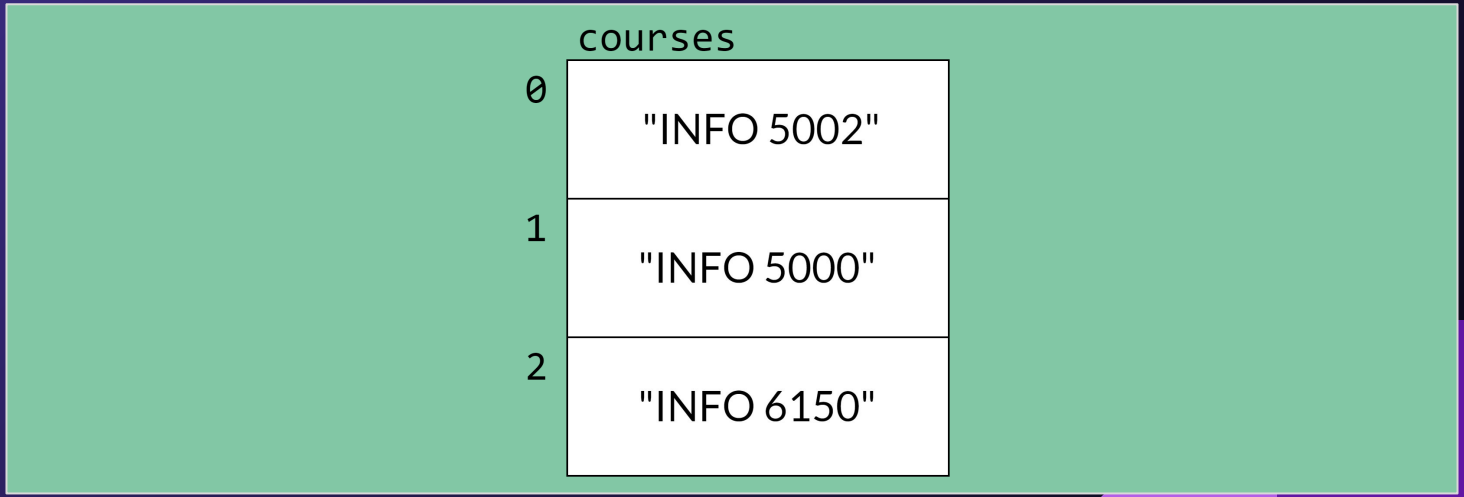
```
const name = 'Fernando';
```



# Arrays

An array can store a list of values of the same type.

```
const courses = ['INFO 5002', 'INFO 5000', 'INFO 6150'];
```



# Arrays

An array can store a list of values of the same type.

Can also do:

```
const courses = [];

// Add a value to the "end" of the list.
courses.push('INFO 5002');
courses.push('INFO 5000');
courses.push('INFO 6150');
```

Arrays in Javascript are **dynamic:** they will grow on demand.

# Arrays

Can ALSO do:

```
const courses = new Array(3); // an array of size 3

// Referring to each item of the list by index number
courses[0] = 'INFO 5002'; // we start at 0!
courses[1] = 'INFO 5000';
courses[2] = 'INFO 6150';
```

The number within the brackets is called **index**

# Other operations on arrays

```
courses.shift('INFO 7001'); // insert at the beginning
console.log(courses[0]); // returns 'INFO 7001'
console.log(courses.length); // 4

const last = courses.pop(); // returns the last item and removes it from the array
console.log(last); // returns 'INFO 6150'
console.log(courses.length); // 3
```

Quiz: How to display the **last** element of the array (without removing it)?

# A note on array.length

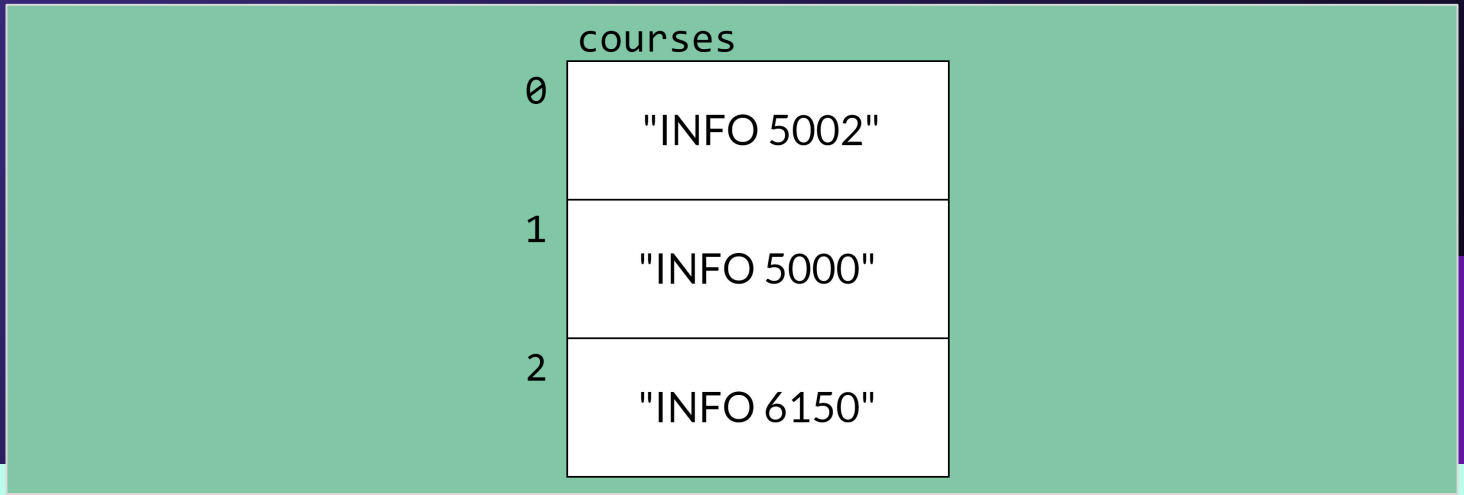
```
const courses = ['INFO 5002', 'INFO 5000', 'INFO 6150'];
console.log(courses.length); // 3
```

What is the index of the last element?

# A note on array.length

```
const courses = ['INFO 5002', 'INFO 5000', 'INFO 6150'];
console.log(courses.length); // 3
```

Arrays are zero-based, so the last element has the index `length - 1`.

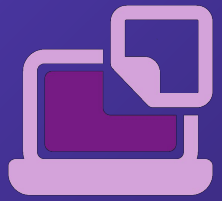


# A note on array.length

```
const courses = ['INFO 5002', 'INFO 5000', 'INFO 6150'];
console.log(courses.length); // 3
```

Arrays are zero-based, so the last element has the index `length - 1`.

```
console.log(courses[courses.length - 1]); // INFO 6150
console.log(courses[courses.length]); // Array out of bounds error
```



Created by HideMaru
from Noun Project

# Creating a to-do list

Store the following list of to-dos in an array:

- Get groceries
- Go to the bank
- Run for president
- Do laundry

Display all of those using `console.log()`.

Icon by HideMaru for Noun Project

# Creating a to-do list

```
const todos = [
    'get groceries',
    'go to the bank',
    'run for president',
    'do laundry',
];

console.log(todos[0]);
console.log(todos[1]);
console.log(todos[2]);
console.log(todos[3]);
```

Imagine you have 100 items. How would you display those?

# Loops

# Loops

Loops are present in most programming languages. They allow you to **repeat** a block of instructions while a **logical condition** is true.

The three basic loops are: `do…while`, `while`, and `for`.

If you're familiar with Java, the syntax will not be new for you :)

# The do...while loop

What would this loop print?

```
let i = 0;
do {
    console.log(i);
    i = i + 1;
} while (i < 3);
```

# The do...while loop

What would this loop print?

```
let i = 0;
do {
    console.log(i);
    i = i + 1;
} while (i < 3);

// 0
// 1
// 2
```

# The do...while loop

What would happen if we never increment the value of `i`?

```
let i = 0;
do {
    console.log(i);
} while (i < 3);
```

# The do...while loop

What happens here? What gets printed?

```
let i = 4;
do {
    console.log(i);
    i = i + 1;
} while (i < 3);
```

# The do...while loop

What happens here? What gets printed?

```
let i = 4;
do {
    console.log(i);
    i = i + 1;
} while (i < 3);

// 4
```

# The while loop

It’s very similar, but the condition gets evaluated at the beginning of each loop.

What gets printed here?

```
let i = 0;
while (i < 3) {
    console.log(i);
    i = i + 1;
}
```

# The while loop

It’s very similar, but the condition gets evaluated at the beginning of each loop.

What gets printed here? Why is this different to the do…while loop?

```
let i = 0;
while (i < 3) {
    console.log(i);
    i = i + 1;
}

// 0
// 1
// 2
```

# The while loop

What gets printed here? Why?

```
let i = 3;
while (i < 3) {
    console.log(i);
    i = i + 1;
}
```

# The for loop

Cycles with this structure are so common that the for() loop was created as an abbreviated form.

It has this shape:

```
for (initialization; condition; increase)
```

that get executed in this order:

1. Initialization (only once);
2. Contents of loop;
3. Increase;
4. Condition is evaluated; if true, go back to 2; if false, end

# The for loop

How do we create the same loop we've been working on, with a `for()`?

# The for loop

How do we create the same loop we've been working on, with a `for()`?

```
for (let i = 0; i < 3; i = i + 1) {
    console.log(i);
}
```

How does this execute?

# The for loop

All parts of a `for()` loop are optional (but good practice to include).

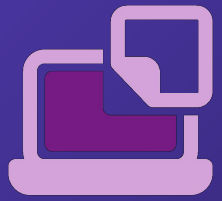
```
for (let i = 0; i < 3;) {
    console.log(i);
    i = i + 1;
}
```

Is this the same as the previous example?  Why or why not?

# The for loop

Here we've initialized to 3. What will happen?

```
for (let i = 3; i < 3; i = i + 1) {
    console.log(i);
}
```



Created by HideMaru
from Noun Project

# Traversing an array

First, try it yourself. How do we display the contents of an array of any size, using a loop?

Remember:

- You can use `array.length` to know how many elements an array has.

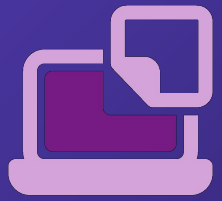
Icon by HideMaru for Noun Project

# Traversing an array

This is a possible answer:

```
const todos = [
    'get groceries',
    'go to the bank',
    'run for president',
    'do laundry',
];
for (let i = 0; i < todos.length; i = i + 1) {
    console.log(todos[i]);
}
```

Why does it work?



Created by HideMaru
from Noun Project

# Traversing an array

How would you display the array contents **in reverse order?**

# Advanced traversing methods: for…of

`for…of`: a special form of for() that loops through each element of an array or any other **iterable**.

You do not get an index variable here. Instead, you get the "current" element in a special variable.

```
const todos = [...];

for (const todo of todos) {
    // the constant "todo" is the "current" element
    // we are visiting
    console.log(todo);
}
```

# Advanced traversing methods: for…of

`for…of`: a special form of for() that loops through each element of an array or any other **iterable**.

You do not get [illegible] ere. Instead, you get the "current" element in a special variable

This means we can apply it to Maps, too, since they are iterable

```
const todos = [...];

for (const todo of todos) {
    // the constant "todo" is the "current" element
    // we are visiting
    console.log(todo);
}
```

# Advanced traversing methods: forEach

`array.forEach()`: send it a **function**. The function will execute once per array element.

The function gets that “current” array element as parameter.

```
const todos = [...];

function eachTodo(todo) {
    console.log(todo);
}

todos.forEach(eachTodo);
```

# Advanced traversing methods: forEach

`array.forEach()`: send it a **function**. The function will execute once per array element.

The function gets that “current” arra ameter.

We will talk more about functions next week!

```
const todos = [...];

function eachTodo(todo) {
    console.log(todo);
}

todos.forEach(eachTodo);
```

# Functions: The basics

# Functions

A function is a piece of a program that can be invoked on its own.

A function has a name*, **inputs** called parameters, and an **output** (a return value).

We put code in functions to avoid repeating it (DRY: Don’t Repeat Yourself).

*Usually

# Functions

```
function sum(a, b) {
    return a + b;
}

const result = 1 + 2;
console.log(result); // 3
```

# Traditional vs. Arrow functions

Two types of functions in Javascript:

- Traditional functions:

```
function handleClick(event) {...}
```

- “Arrow” functions:

```
const handleClick = (event) => {...}
```

# Traditional vs. Arrow functions

Two types of functions in Javascript:

- Traditional functions:

```
function handleClick(event) {...}
```

- “Arrow” functions:

```
const handleClick = (event) => {...}
```

They are equivalent 90% of cases. You call them the same: `handleClick(something)`

There are some differences but we’ll cover them when relevant.

# The “return” statement

Same as with Java, return ends the function and it also allows sending a value back.

```
function sum(a, b) {
    return a + b;
}

const total = sum(3, 5);
// total = 8
```

# Arrow function return abbreviation

With arrow functions you can do something neat: a function that only does one thing can be abbreviated.

For example, this:

```
function sum(a, b) {
    return a + b;
}
```

which can also be written like this:

```
const sum = (a, b) => {
    return a + b;
}
```

Can **also** be written like this:

```
const sum = (a, b) => a + b;
```

Remember this - it may appear in code you see on the internet

# Scope

# Scope

In Javascript, variables and constants exist:

- inside the **block of {} characters** where they are created, and
- inside all blocks inside this one.

This is called their **scope.** It’s an important concept to follow, since it can be the cause of many headaches when writing functions.

# Scope example

Where does the node  constant exist?

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    if (node !== null) {
        node.innerHTML = ""; ⇐ is this okay? Yes, it is - node exists here as well
    }
}
```

# Scope example

Where does the node  constant exist?

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    if (node !== null) {
        node.innerHTML = ""; ⇐ is this okay? Yes, it is - node exists here as well
    }
}
```

# Scope example

Let's say we want to output whether we deleted something or not. We create a `flag` constant for this.

Will this work?

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    if (node !== null) {
        const flag = true;
        node.innerHTML = "";
    }
    if (flag) {
        console.log("We deleted a node");
    } else {
        console.log("We did not delete a node");
    }
}
```

# Scope example

Let’s say we want to output whether we deleted something or not. We create a `flag` constant for this.

Will this work?

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    if (node !== null) {
        const flag = true;
        node.innerHTML = "";
    }
    if (flag) { <= won’t work as expected; our “flag” does not exist here!
        console.log("We deleted a node");
    } else {
        console.log("We did not delete a node");
    }
}
```

How can we fix it?

# Scope example

We need to declare this as a variable, and put it in the “parent” scope instead.

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    let flag = false;
    if (node !== null) {
        flag = true; // changing the value of the parent’s flag variable
        node.innerHTML = "";
    }
    if (flag) {
        console.log("We deleted a node");
    } else {
        console.log("We did not delete a node");
    }
}
```

# Scope example

This works because the scope of flag includes both inside and outside the `if {}`.

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    let flag = false;
    if (node !== null) {
        flag = true; // changing the value of the parent’s flag variable
        node.innerHTML = "";
    }
    if (flag) {
        console.log("We deleted a node");
    } else {
        console.log("We did not delete a node");
    }
}
```

# Scope example

This works because the scope of flag includes both inside and outside the `if {}`.

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    let flag = false;
    if (node !== null) {
        flag = true; // changing the value of the pa
        node.innerHTML = "";
    }
    if (flag) {
        console.log("We deleted a node");
    } else {
        console.log("We did not delete a node");
    }
}
```

Note: `flag` starts existing in this line, and not before! This is true of both “let” and “const”.

# Scope shadowing (DO NOT DO THIS)

Be careful: it’s possible to define a variable or constant with the same name in a “child” block.

What is happening here?

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    let flag = false;
    if (node !== null) {
        const flag = true;
        node.innerHTML = "";
    }
    if (flag) {
        console.log("We deleted a node");
    } else {
        console.log("We did not delete a node");
    }
}
```

# Scope shadowing (DO NOT DO THIS)

This will never work!

- There are **two** `flag` with different scopes.
- We set the inner `flag` to `true`, but the outer `flag` never changes value!

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    let flag = false;
    if (node !== null) {
        const flag = true;
        node.innerHTML = "";
    }
    if (flag) {
        console.log("We deleted a node");
    } else {
        console.log("We did not delete a node");
    }
}
```

# Scope shadowing (DO NOT DO THIS)

This will never work!

- There are **two** `flag` with different scopes.
- We set the inner `flag` to `true`, but the outer `flag` never changes value!

```
function resetNode(nodeName) {
    const node = document.querySelector(nodeName);
    let flag = false;
    if (node !== null) {
        const flag = true;
        node.innerHTML = "";
    }
    if (flag) {
        console.log("We deleted a node");
    } else {
        console.log("We did not delete a node");
    }
}
```

# Scope also applies to functions

It is possible to write a function inside another one. In this case, the inner function only exists inside the outer one.

This may sound weird, but it’s useful in certain cases.

When writing a class, whatever we declare inside the class (variables and functions) only exist inside the class. It’s the same concept.

```
class Triangle {
    function getArea() {...}
}

getArea(); // won't work

const triangle = new Triangle();
triangle.getArea(); // but this will work
```

# The global scope

When you write a function, variable or constant outside anything else, it lives in the **global scope.** All of the functions we've written so far are in the global scope.

The global scope is accessible by everything else.

- Things such as `document` and `window` exist in the global scope.

# The global scope

For small programs it's okay to use it, but in general **it's not a good idea to put variables and constants in the global scope:**

- As the program grows, the global scope can get polluted with objects that don't need to be there.

To avoid this, we can use classes (OOP) and functions.

- In general, it's a good idea to put code inside functions, at least.

There are other techniques such as namespaces and modules - we *may* explore these later.

- Frameworks such as React, Angular, etc. handle this automatically

# Appendix: Making good functions

# Making a good function

- Single Responsibility: Make sure it only does one thing, if possible.
- Don't Repeat Yourself: Make generic functions.
  - Define an input and an output.
- Make **idempotent** functions that always produce the same set of outputs for a given set of inputs.
  - This is not always possible (for example, if there are random elements in the function, or side effects need to occur).
- Figure out **edge cases** and cover them.
  - How can the function fail? How are we preventing it from failing, or handing its failure?

# Making a good function

- Single Responsibility: Make sure it only does one thing, if possible.
- Don't Repeat Yourself: Make generic functions.
  - Define an input and an output.
- Make i**dempotent** functions that always produce the same set of outputs for a given set of inputs.
  - This is not always possible (for example, if there are random elements in the function, or side effects need to occur).
- Figure out **edge cases** and cover them.
  - How can the function fail? How are we preventing it from failing, or handing its failure?

# The Single Responsibility Principle

A good function should follow the Single Responsibility Principle: **it should do one thing, and only one thing.**

(This is one of the five SOLID principles. It applies to classes, but also to functions.)

□💬Claude, what is the Single Responsibility Principle?

□💬Claude, give me an example.

# Does this function do only one thing?

```
function processUserOrder(user, items) {
  let total = 0;
  for (let item of items) {
    total += item.price * item.quantity;
  }

  if (user.isPremium) {
    total *= 0.9;
  }

  const receipt = `Customer: ${user.name}\nTotal: $${total.toFixed(2)}\nDate: ${new Date().toLocaleDateString()}`;

  console.log(`Sending email to ${user.email}: ${receipt}`);

  return receipt;
}
```

# No; it does many things:

```
function processUserOrder(user, items) {
  // Responsibility 1: Calculate total
  let total = 0;
  for (let item of items) {
    total += item.price * item.quantity;
  }

  // Responsibility 2: Apply discount
  if (user.isPremium) {
    total *= 0.9;
  }

  // Responsibility 3: Format receipt
  const receipt = `Customer: ${user.name}\nTotal: $${total.toFixed(2)}\nDate: ${new Date().toLocaleDateString()}`;

  // Responsibility 4: Send email
  console.log(`Sending email to ${user.email}: ${receipt}`);

  return receipt;
}
```

# Breaking functions down

When a function gets too long, it may be time to break it down.

When to do it?

- When it’s so long, it’s difficult to read and understand.
- When it does too many things, especially if they don’t have a clear relation.
- When part of the function gets repeated somewhere else.

# Breaking functions down

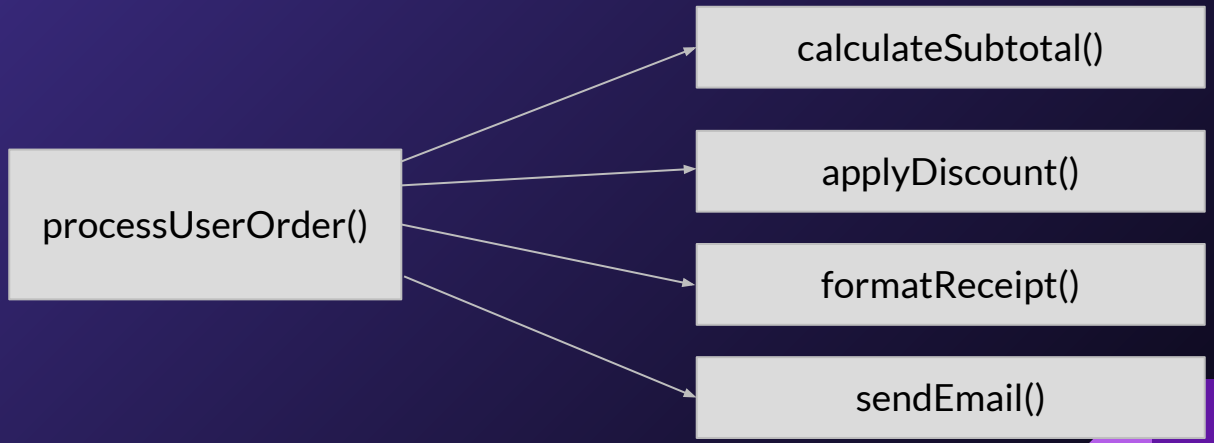
When a function gets too long, it may be time to break it down.

When to do it?

- When it's so long, it's difficult to read and understand.
- When it does too many things, especially if they don't have a clear relation.
- **When part of the function gets repeated somewhere else -> modularity**

# Breaking functions down

Let's break down our `processUserOrder` function so that it does not do too many unrelated things. Let's have one function per responsibility:




(by the way, this is a **box diagram**)

# Creating the sub-functions

We can start by creating the function definitions.

Let's also write quick documentation on what they will do. This format is called JSDoc - it's easy to learn and if you use it, most IDEs will recognize it and give you help on hover.

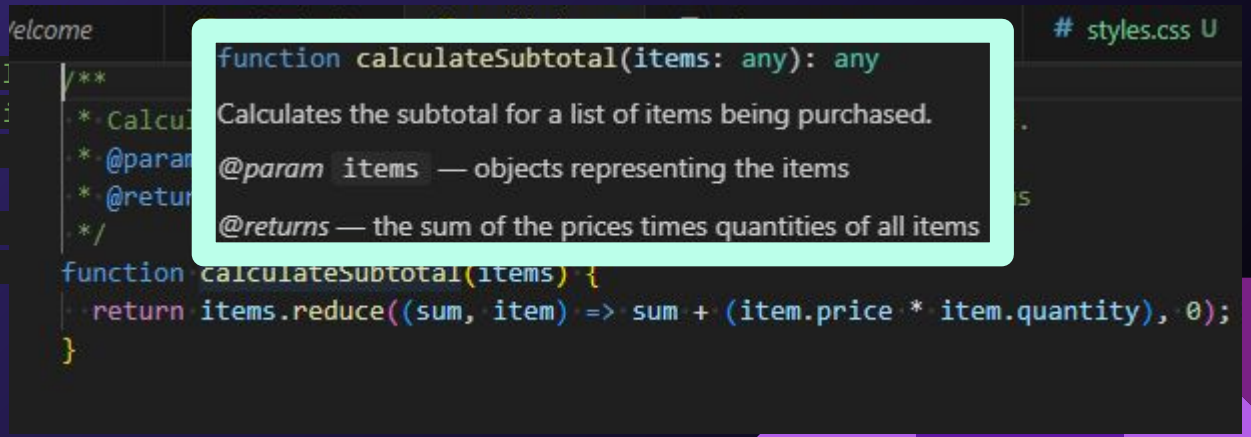
```
/**
 * Calculates the subtotal for a list of items being purchased.
 * @param items objects representing the items
 * @returns the sum of the prices times quantities of all items
 */
function calculateSubtotal(items) {
  return items.reduce((sum, item) => sum + (item.price * item.quantity), 0);
}
```

# Creating the sub-functions

We can start by creating the function definitions.

Let's also write quick documentation on what they will do. This format is called JSDoc - it's easy to learn and if you use it, most IDEs will recognize it and give you help on hover.

```
/**
 * Calculates the subtotal for a l
 * @param items objects representi
 * @returns the sum of the prices
 */
function calculateSubtotal(items)
  return items.reduce((sum, item)
}
```



# ...and now we can do this!

```
function processUserOrder(user, items) {
  // First, sum up the subtotal of the items
  const subtotal = calculateSubtotal(items);
```

which reads more clearly, because **it's plain English.**

Also, because it says **what it does**, and not necessarily how it does it.
(This means: if we need to change how to calculate this validation, we can do it without affecting the main function)

# The whole set of SRP-compliant functions

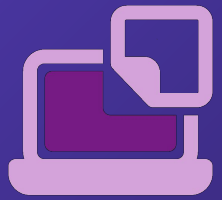
```
function calculateSubtotal(items) {
  return items.reduce((sum, item) => sum +
(item.price * item.quantity), 0);
}

function applyDiscount(total, user) {
  return user.isPremium ? total * 0.9 : total;
}

function formatReceipt(user, total) {
  return `Customer: ${user.name}\nTotal:
$${total.toFixed(2)}\nDate: ${new
Date().toLocaleDateString()}`;
```

```
function sendEmail(email, message) {
  console.log(`Sending email to ${email}:
${message}`);
}

function processUserOrder(user, items) {
  const subtotal = calculateSubtotal(items);
  const total = applyDiscount(subtotal, user);
  const receipt = formatReceipt(user, total);
  sendEmail(user.email, receipt);
  return receipt;
}
```



Created by HideMaru
from Noun Project

# Practice time!

- Download the three files here: https://github.com/sgenius/web-samples/tree/main/info-6150/05-usability-forms-functions-layouts/js4-single-responsibility
- Open the Javascript file. It contains a single function.
  - Why does it break the Single Responsibility Principle?
- Open the HTML file in your browser so that you can see it at work.

Break down the function so that it follows the Single Responsibility Principle.

- Note: The function must still work!

You have 10 minutes.

Icon by HideMaru for Noun Project

# Making a good function

- Single Responsibility: Make sure it only does one thing, if possible.
- Don’t Repeat Yourself: Make generic functions.
  - Define an input and an output.
- Make i**dempotent** functions that always produce the same set of outputs for a given set of inputs.
  - This is not always possible (for example, if there are random elements in the function, or side effects need to occur).
- Figure out **edge cases** and cover them.
  - How can the function fail? How are we preventing it from failing, or handing its failure?

# The DRY principle

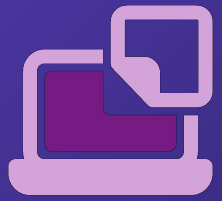
**DRY means Don’t Repeat Yourself.**

In other words: try not to write the same code many times. Instead, put it in a function and call the function.

Why?

- More code written = more code to  maintain and debug.
- It’s possible to make mistakes when repeating the code.
- It creates code that is more difficult to understand.

□💬Claude, show me an example of DRY code.



Created by HideMaru
from Noun Project

# Practice time!

Copy the Claude-generated code in the next slide. How does it violate the DRY principle? Rewrite it to fix.

Icon by HideMaru for Noun Project

```javascript
// BAD CODE - Violates DRY principle!

const products = [
  { name: 'Laptop', price: 999, category: 'Electronics' },
  { name: 'Phone', price: 699, category: 'Electronics' },
  { name: 'Tablet', price: 499, category: 'Electronics' },
  { name: 'Headphones', price: 199, category: 'Electronics' }
];

// Calculating total for laptops
let laptopTotal = 0;
for (let i = 0; i < products.length; i++) {
  if (products[i].name === 'Laptop') {
    laptopTotal = laptopTotal + products[i].price;
  }
}
console.log('Total for Laptops: $' + laptopTotal);

// Calculating total for phones
let phoneTotal = 0;
for (let i = 0; i < products.length; i++) {
  if (products[i].name === 'Phone') {
    phoneTotal = phoneTotal + products[i].price;
  }
}
console.log('Total for Phones: $' + phoneTotal);

// Calculating total for tablets
let tabletTotal = 0;
for (let i = 0; i < products.length; i++) {
  if (products[i].name === 'Tablet') {
    tabletTotal = tabletTotal + products[i].price;
  }
}
console.log('Total for Tablets: $' + tabletTotal);

// Applying 10% discount to laptop
let discountedLaptop = products[0].price - (products[0].price * 0.10);
console.log('Laptop with discount: $' + discountedLaptop);

// Applying 10% discount to phone
let discountedPhone = products[1].price - (products[1].price * 0.10);
console.log('Phone with discount: $' + discountedPhone);

// Applying 10% discount to tablet
let discountedTablet = products[2].price - (products[2].price * 0.10);
console.log('Tablet with discount: $' + discountedTablet);
```

# Making a good function

- Make sure it only does one thing, if possible.
- Try to make generic functions (Don't Repeat Yourself).
  - Define an input and an output.
- The best functions are **idempotent:** they always produce the same set of outputs for a given set of inputs.
  - This is not always possible (for example, if there are random elements in the function, or side effects need to occur).
- Figure out **edge cases** and cover them.
  - How can the function fail? How are we preventing it from failing, or handing its failure?

# Making a good function

- Make sure it only does one thing, if possible.
- Try to make generic functions (Don't Repeat Yourself).
  - Define an input and an output.
- The best functions are **idempotent:** they always produce the same set of outputs for a given set of inputs.
  - This is not always possible (for example, if there are random elements in the function, or side effects need to occur).
- Figure out **edge cases** and cover the
  - How can the function fail? How are we preve

Most functions are idempotent.
We will study side effects later.

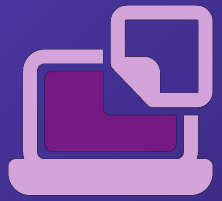
# Making a good function

- Make sure it only does one thing, if possible.
- Try to make generic functions (Don't Repeat Yourself).
  - Define an input and an output.
- The best functions are **idempotent:** they always produce the same set of outputs for a given set of inputs.
  - This is not always possible (for example, if there are random elements in the function, or side effects need to occur).
- Figure out **edge cases** and cover them.
  - How can the function fail? How are we preventing it from failing, or handing its failure?

# How to find edge cases?

- “Play” with your program: use it in ways you haven’t tested before
- Even better: ask someone else to use it.

□💬Claude, how do I find edge cases?



Created by HideMaru
from Noun Project

# Practice time!

Copy these Claude-generated functions. Look for all ways in which they can break. Then, write code to fix them, if needed.

```
// Function 1: Check if a string is a palindrome
function isPalindrome(str) {
  const reversed = str.split('').reverse().join('');
  return str === reversed;
}

// Function 2: Calculate the average of numbers in an array
function calculateAverage(numbers) {
  const sum = numbers.reduce((acc, num) => acc + num, 0);
  return sum / numbers.length;
}
```

# Appendix:
# Keyed collections

# Map

Map is an object with key-value pairs. Keys and values can be of any type.

There can only be **one value for each key**.

[Try the Mozilla demo here.]

```
const map = new Map();

map.set("a", 1);
map.set("b", 2);
map.set("c", 3);

console.log(map.get("a"));
// Expected output: 1

map.set("a", 97);

console.log(map.get("a"));
// Expected output: 97

console.log(map.size);
// Expected output: 3

map.delete("b");

console.log(map.size);
// Expected output: 2
```

Source: Mozilla

# Map

Map is an object with
Keys and values can be of any type.

There can only be **one value for each key**.

[Try the Mozilla demo here.]

Use set() to add a key-value pair.

```
1 const map = new Map();
2
3 map.set("a", 1);
4 map.set("b", 2);
5 map.set("c", 3);
6
7 console.log(map.get("a"));
8 // Expected output: 1
9
10 map.set("a", 97);
11
12 console.log(map.get("a"));
13 // Expected output: 97
14
15 console.log(map.size);
16 // Expected output: 3
17
18 map.delete("b");
19
20 console.log(map.size);
21 // Expected output: 2
```

Source: Mozilla

# Map

Map is an object with key-value pairs. Keys and values can be of any type.

There can only be **one value for each key**.

[Try the Mozilla demo here.]

Use delete() to remove a key-value pair; the argument is the key.

```
1  const map = new Map();
2
3  map.set("a", 1);
4  map.set("b", 2);
5  map.set("c", 3);
6
7  console.log(map.get("a"));
8  // Expected output: 1
9
10 map.set("a", 97);
11
12 console.log(map.get("a"));
13 // Expected output: 97
14
15 console.log(map.size);
16 // Expected output: 3
17
18 map.delete("b");
19
20 console.log(map.size);
21 // Expected output: 2
```

Source: Mozilla

# More on Maps

While similar to regular objects, they have some differences:

- They are **iterable**; an object is not.
- When looping through a Map, it will produce its items in the order in which they were added to it.
- They do not contain any keys by default in their prototype. (In some cases, the Object prototype methods' names can collide with your own names.)
- Keys can be of any type.
- They are more performant in additions and removals.

# More on Maps

More about this in the “loops” section

While similar t s, they have some differences:

- They are **iterable**; an object is not.
- When looping through a Map, it will produce its items in the order in which they were added to it.
- They do not contain any keys by default in their prototype. (In some cases, the Object prototype methods’ names can collide with your own names.)
- Keys can be of any type.
- They are more performant in additions and removals.

# Set

A Set object is a collection of values. The values have no key.

Values in a Set can only appear once.

A Set contains methods for algebra of sets such as `difference()`, `union()` and `isSubsetOf()`.

However, **the method I use the most** is has(): it tells you if a value is in the set or not. It's much faster and simpler than looking for the value in an array or an object.

# Set

A great way to guarantee that values are unique in an array is to make a Set out of your array.

```
> const someArray = [1, 2, 66, 3, 1, 2, 1, 1]
<· undefined
> const someArraySet = new Set(someArray)
<· undefined
> console.log(someArraySet)
  ▸ Set(4) {1, 2, 66, 3}
<· undefined
> const backToArray = [...someArraySet]
<· undefined
> console.log(backToArray)
  ▸ (4) [1, 2, 66, 3]
```

# Set

In a set, use `.size` to know how many elements it has.

```
> someArraySet.size
<· 4
> someArraySet.has(22)
<· false
```